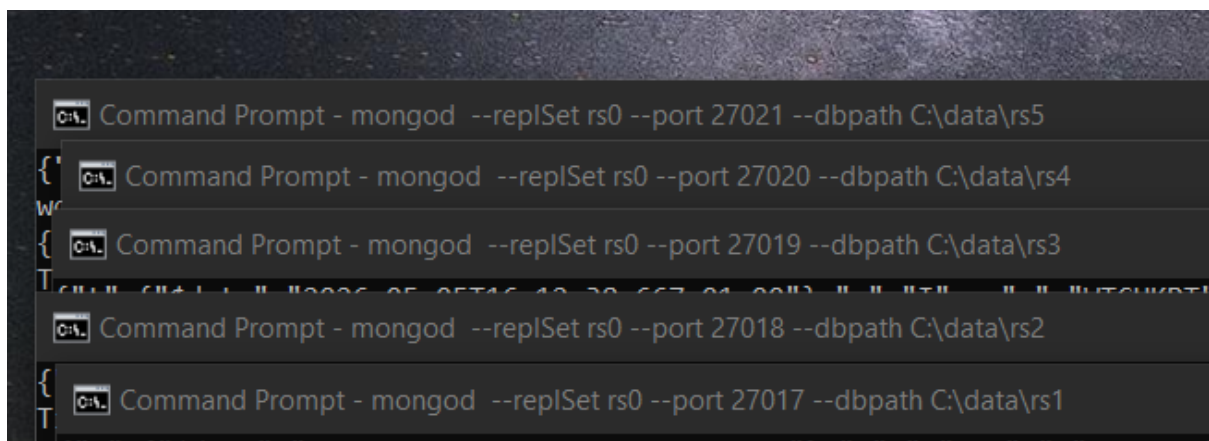


REPLICATION

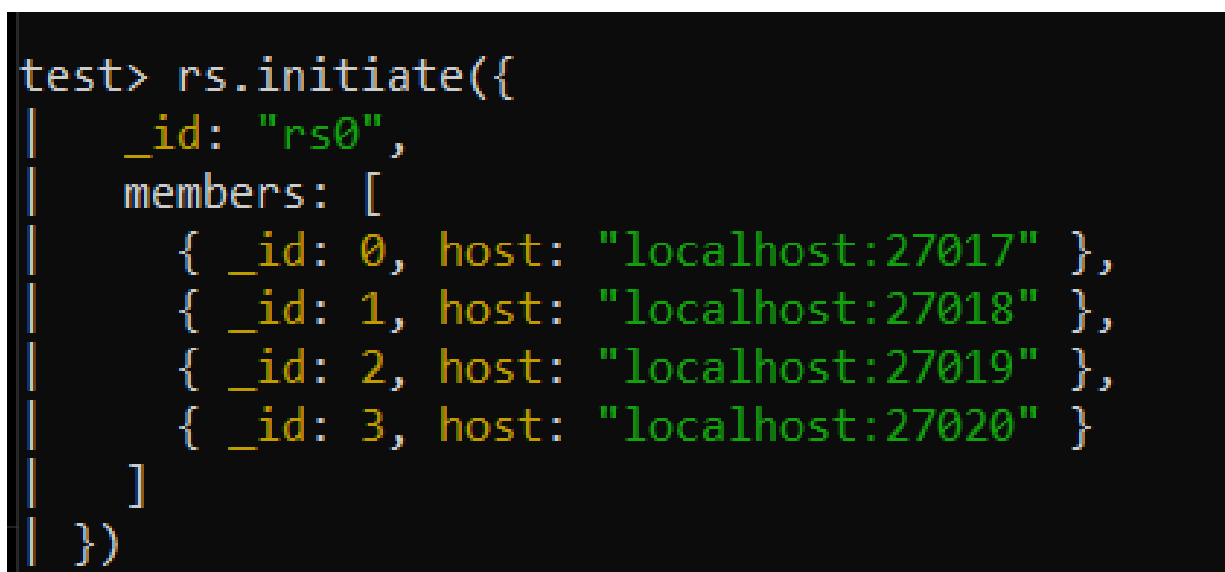
Q1

For this first question a I created a replica set with 5 nodes using MongoDB in a psa architecture, consisting of one primary node, three secondary nodes and one arbiter.

Each node was started on a different port and initialized using the rs.initiate() command. The arbiter node was added using rs.addArb(), the replica set was verified using rs.status() which showed the correct roles of all nodes. Final, the dataset was imported into the primary node, and the data was automatically replicated to the secondary nodes then imported was confirmed.



```
C:\> Command Prompt - mongod --replSet rs0 --port 27021 --dbpath C:\data\rs5
{
C:\> Command Prompt - mongod --replSet rs0 --port 27020 --dbpath C:\data\rs4
W
{
C:\> Command Prompt - mongod --replSet rs0 --port 27019 --dbpath C:\data\rs3
T
C:\> Command Prompt - mongod --replSet rs0 --port 27018 --dbpath C:\data\rs2
{
C:\> Command Prompt - mongod --replSet rs0 --port 27017 --dbpath C:\data\rs1
T
```



```
test> rs.initiate({
|   _id: "rs0",
|   members: [
|     { _id: 0, host: "localhost:27017" },
|     { _id: 1, host: "localhost:27018" },
|     { _id: 2, host: "localhost:27019" },
|     { _id: 3, host: "localhost:27020" }
|   ]
| })
```

```
rs0 [direct: primary] test> _
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1777993641, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1777993641, i: 1 })
}
```

```
,
members: [
  {
    _id: 0,
    name: 'localhost:27017',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 852,
    optime: { ts: Timestamp({ t: 1777993728, i: 1 }), t: Long('1') },
    optimeDate: ISODate('2026-05-05T15:08:48.000Z'),
    optimeWritten: { ts: Timestamp({ t: 1777993728, i: 1 }), t: Long('1') },
    optimeWrittenDate: ISODate('2026-05-05T15:08:48.000Z'),
    lastAppliedWallTime: ISODate('2026-05-05T15:08:48.152Z'),
    lastDurableWallTime: ISODate('2026-05-05T15:08:48.152Z'),
    lastWrittenWallTime: ISODate('2026-05-05T15:08:48.152Z'),
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1777992997, i: 1 }),
    electionDate: ISODate('2026-05-05T14:56:37.000Z'),
    configVersion: 2,
    configTerm: 1,
    self: true,
    lastHeartbeatMessage: ''
  },
  {
    _id: 1,
    name: 'localhost:27018',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 747,
    optime: { ts: Timestamp({ t: 1777993728, i: 1 }), t: Long('1') },
    optimeDurable: { ts: Timestamp({ t: 1777993728, i: 1 }), t: Long('1') },
    optimeWritten: { ts: Timestamp({ t: 1777993728, i: 1 }), t: Long('1') },
    optimeDate: ISODate('2026-05-05T15:08:48.000Z'),
    optimeDurableDate: ISODate('2026-05-05T15:08:48.000Z'),
    optimeWrittenDate: ISODate('2026-05-05T15:08:48.000Z'),
    lastAppliedWallTime: ISODate('2026-05-05T15:08:48.152Z'),
    lastDurableWallTime: ISODate('2026-05-05T15:08:48.152Z'),
    lastWrittenWallTime: ISODate('2026-05-05T15:08:48.152Z'),
    lastHeartbeat: ISODate('2026-05-05T15:08:53.641Z'),
    lastHeartbeatRecv: ISODate('2026-05-05T15:08:53.642Z'),
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
    syncSourceHost: 'localhost:27017',
    syncSourceId: 0,
    infoMessage: '',
    configVersion: 2,
    configTerm: 1
  },
],
```

```
{
  _id: 2,
  name: 'localhost:27019',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 747,
  optime: { ts: Timestamp({ t: 1777993728, i: 1 }), t: Long('1') },
  optimeDurable: { ts: Timestamp({ t: 1777993728, i: 1 }), t: Long('1') },
  optimeWritten: { ts: Timestamp({ t: 1777993728, i: 1 }), t: Long('1') },
  optimeDate: ISODate('2026-05-05T15:08:48.000Z'),
  optimeDurableDate: ISODate('2026-05-05T15:08:48.000Z'),
  optimeWrittenDate: ISODate('2026-05-05T15:08:48.000Z'),
  lastAppliedWallTime: ISODate('2026-05-05T15:08:48.152Z'),
  lastDurableWallTime: ISODate('2026-05-05T15:08:48.152Z'),
  lastWrittenWallTime: ISODate('2026-05-05T15:08:48.152Z'),
  lastHeartbeat: ISODate('2026-05-05T15:08:53.698Z'),
  lastHeartbeatRecv: ISODate('2026-05-05T15:08:53.805Z'),
  pingMs: Long('0'),
  lastHeartbeatMessage: '',
  syncSourceHost: 'localhost:27017',
  syncSourceId: 0,
  infoMessage: '',
  configVersion: 2,
  configTerm: 1
},
{
  _id: 3,
  name: 'localhost:27020',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 747,
  optime: { ts: Timestamp({ t: 1777993728, i: 1 }), t: Long('1') },
  optimeDurable: { ts: Timestamp({ t: 1777993728, i: 1 }), t: Long('1') },
  optimeWritten: { ts: Timestamp({ t: 1777993728, i: 1 }), t: Long('1') },
  optimeDate: ISODate('2026-05-05T15:08:48.000Z'),
  optimeDurableDate: ISODate('2026-05-05T15:08:48.000Z'),
  optimeWrittenDate: ISODate('2026-05-05T15:08:48.000Z'),
  lastAppliedWallTime: ISODate('2026-05-05T15:08:48.152Z'),
  lastDurableWallTime: ISODate('2026-05-05T15:08:48.152Z'),
  lastWrittenWallTime: ISODate('2026-05-05T15:08:48.152Z'),
  lastHeartbeat: ISODate('2026-05-05T15:08:53.662Z'),
  lastHeartbeatRecv: ISODate('2026-05-05T15:08:53.781Z'),
  pingMs: Long('0'),
  lastHeartbeatMessage: '',
  syncSourceHost: 'localhost:27017',
  syncSourceId: 0,
  infoMessage: '',
  configVersion: 2,
  configTerm: 1
}
```

```
{
  _id: 4,
  name: 'localhost:27021',
  health: 1,
  state: 7,
  stateStr: 'ARBITER',
  uptime: 93,
  lastHeartbeat: ISODate('2026-05-05T15:08:53.687Z'),
  lastHeartbeatRecv: ISODate('2026-05-05T15:08:53.703Z'),
  pingMs: Long('0'),
  lastHeartbeatMessage: '',
  syncSourceHost: '',
  syncSourceId: -1,
  infoMessage: '',
  configVersion: 2,
  configTerm: 1
}
```

```
rs0 [direct: primary] test> use employeesdb
switched to db employeesdb
rs0 [direct: primary] employeesdb> db.employee_burnout.count
0
rs0 [direct: primary] employeesdb> db.employee_burnout.count
849999
rs0 [direct: primary] employeesdb> _
```

Q2

Failover

In a replica set an automatic failover occurs when the primary node becomes unavailable meaning that the secondary node must detect the failure and hold an election. This means that a new primary node is selected from the secondary nodes allowing the system to continue operating without manual intervention

the arbiter does not store data, but it participates in elections. This helps the replica set maintain an odd number of voting members, making it easier to elect a new primary if the current primary fails.

Oplog

The Oplog is a special collection used to record all write operations performed on the primary node. Secondary nodes use the Oplog to replicate data, now in order to increase the Oplog size to 25GB, the mongod process must be started with `--oplogsize` parameter set to 25600mb

```
C:\> Command Prompt - mongod --replSet rs0 --port 27021 --dbpath C:\data\rs5 --oplogSize 25600
```

```
C:\> Command Prompt - mongod --replSet rs0 --port 27020 --dbpath C:\data\rs4 --oplogSize 25600
```

```
C:\> Command Prompt - mongod --replSet rs0 --port 27019 --dbpath C:\data\rs3 --oplogSize 25600
```

```
C:\> Command Prompt - mongod --replSet rs0 --port 27018 --dbpath C:\data\rs2 --oplogSize 25600
```

```
C:\> Command Prompt - mongod --replSet rs0 --port 27017 --dbpath C:\data\rs1 --oplogSize 25600
```

Q3

For this question we need to write operations and read operations by making sure writes are confirmed by 2 data nodes and make sure it reads return the most recent data

Write operation:

To ensure that write operations are completed with two data bearing members, the write concern was set to w:2. This ensures that a write is only acknowledge after it has been written to the primary node and at least one secondary node

```
operationTime: Timestamp({ t: 1778000803, i: 1 })
}
rs0 [direct: primary] employeesdb> db.adminCommand({
  setDefaultRWConcern: 1,
  defaultWriteConcern: { w: 2 }
})
{
  defaultReadConcern: { level: 'local' },
  defaultWriteConcern: { w: 2, wtimeout: 0 },
  updateOpTime: Timestamp({ t: 1778001063, i: 1 }),
  updateWallClockTime: ISODate('2026-05-05T17:11:11.643Z'),
  defaultWriteConcernSource: 'global',
  defaultReadConcernSource: 'implicit',
  localUpdateWallClockTime: ISODate('2026-05-05T17:11:11.657Z'),
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778001071, i: 2 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778001071, i: 2 })
}
rs0 [direct: primary] employeesdb> db.getSiblingDB("admin").runCommand({ getDefaultRWConcern: 1 })
{
  defaultReadConcern: { level: 'local' },
  defaultWriteConcern: { w: 2, wtimeout: 0 },
  updateOpTime: Timestamp({ t: 1778001063, i: 1 }),
  updateWallClockTime: ISODate('2026-05-05T17:11:11.643Z'),
  defaultWriteConcernSource: 'global',
  defaultReadConcernSource: 'implicit',
  localUpdateWallClockTime: ISODate('2026-05-05T17:11:11.657Z'),
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778001143, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778001143, i: 1 })
}
rs0 [direct: primary] employeesdb>
```

Read operation:

In order to ensure the read operation returns the most recent data, the read concern was set to majority. This guarantees that the data has been replicated to the majority of nodes before being read.

To also avoid stale reads from secondaries, reads are directed to the primary using the default read preference primary. Finally, the majority read concern ensures that the returned data has been majority committed.

```
rs0 [direct: primary] employeesdb> db.adminCommand({
|   setDefaultRWConcern: 1,
|   defaultReadConcern: { level: "majority" }
| })
{
  defaultReadConcern: { level: 'majority' },
  defaultWriteConcern: { w: 2, wtimeout: 0 },
  updateOpTime: Timestamp({ t: 1778001333, i: 1 }),
  updateWallClockTime: ISODate('2026-05-05T17:15:42.790Z'),
  defaultWriteConcernSource: 'global',
  defaultReadConcernSource: 'global',
  localUpdateWallClockTime: ISODate('2026-05-05T17:15:42.798Z'),
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778001342, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778001342, i: 1 })
}
```

Q4

For this question we need to find out how to support geographically local read operations in the replica set

Essentially, how to make MongoDB return data from the closest server geographically. This is done using Read Preference (readPreference: "nearest")

Basically, geographically local read operations can be supported using the read preference setting, by settings the read preference to nearest, MongoDB automatically selects the node with the lowest network latency, which is typically the closest node to the user. This improves performance by reducing response time.

Example

```
rs0 [direct: primary] employeesdb> db.employee_burnout.find().readPref("nearest")
[
  {
    _id: ObjectId('69fa0e9dc358dc73b49c994c'),
    employee_id: 'SYN_00000000',
    role: ' ',
    job_level: 'Mid',
    department: 'Research & Development',
    tenure_months: 169,
    salary: 79704.57905949898,
    performance_score: 0.6324820831281136,
    satisfaction_score: 0.6237460118062441,
    workload_score: 0.7581167715492352,
    team_sentiment: 0.6623349378393414,
    recent_feedback: 'Close to home and good shift time given Slow in agility\r\n' +
      'Hierarchical structure\r\n' +
      'Strong Bureaucracy Needs a bigger focus on inclusion (as does 99.9% of every corporation).',
    communication_patterns: 'Exhibits communication style typical of a ChangeResistor persona.',
    project_completion_rate: 0.5241872767932165,
    overtime_hours: 0,
    training_participation: 0.04437343678133422,
    collaboration_score: 0.4921306859259787,
    technical_skills: [
      'REST APIs',
      'Python (Pandas, NumPy, SciPy)',
      'Linux/Unix Administration',
      'PHP (Laravel)',
      'R',
      'Machine Learning Theory',
      'Docker',
      'dbt'
    ],
  }
]
```

Q5

For this final question, what its essentially asking is how to remove / shutdown the replica set

Now, in order to remove the replica set, all MongoDB instances must be stopped by shutting down each node. This can be done by terminating each mongod process, optionally, the data directories can be deleted to completely remove the replica set config and data

In this case I opted to just shutdown each node by pressing CTRL + C

```
Millis:10, shutdownStorageEngineMillis:1703, shutdownLocalMetricsMillis:10, shutdownIndexMillis:14, shutdownReplicaSetNode
ExecutorMillis":2,"shutDownOCSPMillis":0,"shutdownTaskTotalMillis":16918}}}}
{"t":{"$date":"2026-05-05T18:39:27.703+01:00"},"s":"I", "c":"CONTROL", "id":23138, "ctx":"consoleTerminate","msg":"S
hutting down","attr":{"exitCode":12}}
C:\Users\reyco>
```